

AI Pioneers

CCA-F | Module 2

Lab 2.1

Designing Reliable Tools: Interfaces, Errors & Selection Control

Scenario: AI Customer-Support Agent for an Online Outdoor-Gear Store

Module	M2 — Reliable Tool Use
Lab type	Hands-on · Self-paced or Instructor-Led
Duration	~45 minutes (hands-on)
Environment	Blue Labs VM · Python 3.9+ · Anthropic SDK
Sections	S1 (Tool Interfaces) · S2 (Structured Errors & Retries) · S3 (Selection Control)

Lab Objectives

By the end of this lab you will be able to:

- **Design tool interfaces that drive correct selection** — write names, descriptions, and typed parameter schemas so the model routes each request to the right tool, using explicit when-to-use / when-NOT-to-use contrast (e.g. keeping “do you sell a tent?” separate from “where is my order?”) without reaching for a bigger model.
- **Return failures as structured data** — wrap a flaky service so it always returns an `isError` / `isRetryable` envelope, then build a loop that retries transient failures (timeout, 503) with backoff and stops immediately on permanent ones (404, 400).
- **Scope a turn with `tool_choice`** — choose between `auto`, `any`, and a forced single tool, picking the narrowest setting that still does the job — and forcing a classifier so a triage step becomes deterministic.

1. Overview

An agent is only as trustworthy as its tools. A model that picks the wrong tool, crashes on a transient error, or wanders off to draft a reply when you needed a classification is not production-ready — no matter how capable it is. This lab builds the three properties that make tool use reliable: tools the model selects correctly (interfaces), errors it can recover from (structured failures), and a way to control exactly which tool runs (selection control). You will build all three around one realistic system — the customer-support agent for NorthPeak Outfitters — where a wrong tool or an unhandled error is something a real customer feels.

SN	Section	What you will build	Core concept
Ex 1	S1 — Tool Interfaces	A weak toolset and a strong toolset run over the same model, with a harness that measures how often each routes six support questions to the correct tool.	Selection reliability is an interface problem. Precise names, descriptions that say when NOT to use a tool, and typed parameters — not a bigger model — are what make the model choose correctly.
Ex 2	S2 — Structured Errors	A tool wrapper that turns a flaky Orders service into an <code>isError</code> / <code>isRetryable</code> envelope, plus a retry loop that backs off on transient errors and stops on permanent ones.	Return failures as data, never exceptions. The envelope lets the loop retry a timeout and stop on a 404 — the model never sees a Python crash.

SN	Section	What you will build	Core concept
Ex 3	S3 — Tool Distribution & Selection Control	A triage step run under auto, any, and a forced tool_choice, showing which mode reliably produces one clean classification per ticket.	tool_choice scopes a turn. Use the narrowest setting that still works; forcing the classifier makes triage deterministic.

2. Scenario

You are building the customer-support agent for NorthPeak Outfitters, a fictional online outdoor-gear store. Customers message the agent all day with two very different kinds of question — “do you sell a four-person tent?” (a catalog question) and “where is my order NP-100245?” (an existing-order question). The agent has a tool for each, and if it confuses them the customer gets nonsense: a catalog search for an order number, or an order lookup for a product that was never bought.

On top of that, the backend the agent depends on is imperfect. The Orders service is flaky — it sometimes times out or returns a 503, and sometimes returns a 404 for an order that does not exist. And before any ticket is routed to a queue, it must be classified into exactly one category, deterministically, with no drafting and no chit-chat.

Your task is to build the parts of this agent that make its tool use trustworthy. It must:

- Pick the right tool for each customer question — catalog search vs. existing-order lookup — driven by the tool interface, not by guesswork.
- Survive a flaky backend: retry transient failures automatically, give up gracefully on permanent ones, and never crash the conversation loop.
- Run a deterministic triage step that always produces exactly one classification per ticket before routing.

Live Example (used across the lab)

```
Question A: "Do you carry a four-person tent?"    -> CATALOG search
(search_products)
Question B: "Where is my order NP-100245?"        -> ORDER lookup
(get_order_status)
Order id format: NP-XXXXXX    (regex ^NP-[0-9]{6}$)    e.g. NP-100245, NP-100311,
NP-100190
```

Why three concepts in one lab?

An agent that touches real systems needs all three at once, and they reinforce each other. The interface (Ex 1) decides which tool fires; the structured-error envelope (Ex 2) decides what happens when that tool fails; and tool_choice (Ex 3) decides whether the model is even allowed to reach for a tool on this turn. Strong interfaces with no error handling still crash; perfect error handling behind a tool the model never selects correctly is wasted. Reliable tool use is the combination.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Introduction to Model Context Protocol	Defining tools with MCP (S1, S2); MCP clients; Implementing a client (S3)	S1, S2, S3
Model Context Protocol: Advanced Topics	JSON message types	S2
Introduction to Agent Skills	What are skills?; Skills vs. other Claude Code features (supporting)	S1, S3

The lab assumes you have completed Lab 1.x and understand the tool-use loop, `stop_reason`, and how `tool_result` blocks are fed back to the model. The instructor will not re-teach these — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open <https://www.bluelabs.studio/>, locate your VM, click Start, then Connect.
- Open a terminal in the VM and verify Python 3.9+:

```
python3 --version
```

- Install the Anthropic SDK if not present:

```
pip install "anthropic>=0.40.0"
```

- Create and enter your working directory:

```
lab-2.1-reliable-tools  
cd lab-2.1-reliable-tools
```

- (Local dev only) put your key in a `.env` file and export it:

```
echo 'ANTHROPIC_API_KEY=sk-ant-...' > .env  
export $(grep -v '^#' .env | xargs)
```

ANTHROPIC_API_KEY & ANTHROPIC_MODEL

On the Blue Labs VM your API key is pre-configured as an environment variable — you do not need to set it manually. If you get an authentication error, run `echo $ANTHROPIC_API_KEY` to verify it is present. Every exercise file reads the model from `ANTHROPIC_MODEL` and defaults to a Sonnet model (`claude-sonnet-4-6`); never hard-code a key in source.

4. Exercises

Work through all three exercises in order — each is one self-contained file you can just run. Suggested timings are shown per exercise. Prioritise understanding over completion; the reflection questions at the end of each exercise are where the learning lands.

Exercise 1: Tool Interfaces (S1) ~15 min

Background

The model never sees your implementation. It chooses a tool from three things only: the name, the description, and the parameter schema. So selection reliability is an interface problem, not a model-size problem — which Exercise 1 proves by running the *same model* over a weak and a strong toolset.

A weak toolset gives the model nothing to disambiguate on: vague names (`search`, `lookup`), overlapping descriptions (“search for stuff”), and loose parameters (a field called `q`). A strong toolset fixes that with three levers:

Lever	Weak	Strong
Name	<code>search</code> , <code>lookup</code>	<code>search_products</code> , <code>get_order_status</code> — object + action, never a vague verb
Description	“Search for stuff in the system.”	Says when to use it AND when NOT to (explicitly defers to the sibling tool).
Parameters	<code>q: string</code>	<code>query</code> , or <code>order_id</code> typed with a pattern of <code>^NP-[0-9]{6}\$</code>

Task

Design a strong toolset of two tools — one that searches the product CATALOG and one that looks up an EXISTING ORDER — then run the harness you built to measure how often the weak and strong sets route six support questions to the correct tool. The harness forces a tool call with `tool_choice={"type": "any"}` so you are measuring which tool is picked, not whether one is.

Step 1 — Create `exercise_1_tool_interfaces.py` and design **STRONG_TOOLS**

You will create this file yourself — including `WEAK_TOOLS`, the six `TEST_CASES`, and the scoring harness, which you build on your own. Your main focus is the strong toolset. A good tool has all three of:

- **A name that describes object + action** — `search_products`, `get_order_status` — not a bare verb like `lookup`.
- **A description that says when to use it AND when not to** — each one should point at the other tool for the case it does not handle.
- **Typed, specific parameters** — spell out the order-id format with a regex pattern so a malformed id is rejected at the schema.

A worked reference toolset:

```
STRONG_TOOLS = [
```

```

{
  "name": "search_products",
  "description": (
    "Search the NorthPeak product CATALOG for items we sell (tents, "
    "sleeping bags, stoves, boots, etc.) by free-text query. Use this for "
    "availability, price, or whether a product exists. Do NOT use this to "
    "check something a customer already bought — for an existing purchase "
    "use get_order_status instead."
  ),
  "input_schema": {
    "type": "object",
    "properties": {
      "query": {"type": "string",
        "description": "Free-text product query, e.g. '4 person
tent'."},
      "max_results": {"type": "integer", "minimum": 1, "maximum": 10},
    },
    "required": ["query"],
  },
},
{
  "name": "get_order_status",
  "description": (
    "Retrieve the status of an EXISTING customer order by its order ID "
    "(shipping status, items, tracking). Use this whenever the customer "
    "gives an order number or references a purchase. Do NOT use this to "
    "browse the catalog — for products use search_products instead."
  ),
  "input_schema": {
    "type": "object",
    "properties": {
      "order_id": {"type": "string",
        "description": "Order ID in the format 'NP-XXXXXX'.",
        "pattern": "^NP-[0-9]{6}$"},
    },
    "required": ["order_id"],
  },
},
]

```

Step 2 — Run and compare routing

```
python exercise_1_tool_interfaces.py
```

The harness runs all six questions against the weak set, then the strong set, printing OK / MISS per question and a total. The weak set will misroute several cases; the strong set should route all six correctly. Same model, same questions — only the interface changed.

Reflection Questions

- The weak and strong runs use the same model. If accuracy jumps from (say) 3/6 to 6/6, what does that tell you about where tool-selection reliability actually comes from — and what does it tell you about “just use a bigger model” as a fix?
- The strong order tool adds "pattern": "^NP-[0-9]{6}\$" to order_id. What does that pattern buy you beyond helping the model route — what should happen to a malformed id like 100245 at call time?

- Every strong description ends with “Do NOT use this ... use the other tool instead.” Why is that explicit negative contrast more reliable than two good positive descriptions that leave the boundary implicit?

Exercise 2: Structured Errors & Retries (S2) ~15 min

Background

NorthPeak's Orders service is flaky. Some failures are transient — a timeout (504), a rate-limit (429), a brief 503 — and retrying usually works. Others are permanent — a 404 (no such order) or a 400 (malformed id) — and retrying is pointless, just slower.

If the tool raises an exception, the agentic loop crashes and the model never gets to reason about the failure. The fix is to return failures as data. Every call returns a structured envelope:

```
success -> {"isError": False, ...order fields}
failure -> {"isError": True, "isRetryable": <bool>, "status": <int>, "error": <msg>}
```

A small retry loop then does the obvious thing — retry while isRetryable, with backoff and a cap; stop immediately on a permanent error. “Retry a timeout, but stop on a 404.”

Outcome	isRetryable	Loop behaviour
408 / 429 / 500 / 502 / 503 / 504	True	Wait (exponential backoff), retry, up to the attempt cap.
400 (malformed id)	False	Stop immediately; the model asks for a valid id.
404 (not found)	False	Stop immediately; the model reports the order was not found.
success	n/a	Return the order fields to the model.

Task

Create `exercise_2_structured_errors.py` so the tool always returns the structured envelope and the loop retries only retryable errors. Build the mock service, the failure-injection helper, the tool definition, and the agent turn yourself, then implement the two functions in the middle.

Step 1 — Implement `call_order_tool` and `run_with_retry`

`call_order_tool` wraps the raw service and converts a `ServiceError` into a structured dict — it must never raise:

```
RETRYABLE = {408, 429, 500, 502, 503, 504}

def call_order_tool(order_id):
    try:
        data = orders_service(order_id)
        return {"isError": False, **data}
    except ServiceError as err:
        return {
            "isError": True,
            "isRetryable": err.status in RETRYABLE,
```

```

        "status": err.status,
        "error": err.message,
    }

def run_with_retry(order_id, max_attempts=4):
    delay = 0.2
    for attempt in range(1, max_attempts + 1):
        result = call_order_tool(order_id)
        if not result["isError"]:
            return result
        if result["isRetryable"] and attempt < max_attempts:
            time.sleep(delay)
            delay *= 2          # exponential backoff
            continue
        return result          # permanent, or out of attempts -> stop

```

Step 2 — Offline self-check (no API key needed)

Before spending any tokens, prove the envelope is correct in isolation: a good id succeeds, a 404 is a non-retryable error, and a queued 503 is a retryable error.

```
python exercise_2_structured_errors.py --check
```

Step 3 — Run the live agent over three failure shapes

```
python exercise_2_structured_errors.py
```

Watch the loop behave differently per case: (A) NP-100245 times out once (504), the loop waits and retries, the second attempt succeeds and the agent answers normally; (B) NP-999999 returns 404 — no retry, the agent says the order was not found; (C) 100245 is malformed (400) — no retry, the agent asks for a correctly formatted id. The model never sees a crash, only clean tool_result data with is_error set.

Reflection Questions

- The tool returns errors as data instead of raising. What specifically breaks if a tool raises a Python exception mid-loop — and why can't the model recover from that the way it recovers from a tool_result marked is_error?
- run_with_retry uses exponential backoff (0.2s → 0.4s → 0.8s) and a hard cap of four attempts. What goes wrong in production if you drop the cap? What goes wrong if you keep the cap but drop the backoff?
- 404 and 400 are both non-retryable but mean different things to the customer. Should the agent phrase them identically? Sketch how the structured error could carry enough information for the model to respond differently to “not found” vs “malformed id.”

Exercise 3: Selection Control with tool_choice (S3) ~15 min

Background

Every incoming support ticket must first be classified into exactly one routing category before anything else happens. That triage step has to be deterministic: no chit-chat, no drafting a reply, just a classify_ticket call. tool_choice is the dial that scopes what the model may do on a turn:

tool_choice	What the model may do on this turn
<code>{"type": "auto"}</code>	May answer in plain text, pick any tool, or pick no tool. Least constrained.
<code>{"type": "any"}</code>	Must call SOME tool — but it chooses which one (it can pick the wrong tool).
<code>{"type": "tool", "name": "classify_ticket"}</code>	Must call exactly that tool. Deterministic — what a routing step needs.

To make auto and any visibly drift, the agent is given a second tool — `draft_customer_reply` — that the model can wander toward. Only the forced mode guarantees a clean `classify_ticket` call on turn one for every ticket.

Task

Create `exercise_3_tool_choice.py`, fill in the three `tool_choice` values, run the four sample tickets under each mode, and observe which mode reliably produces a classification every time. Then apply the rule: use the narrowest `tool_choice` that still lets the step do its job.

Step 1 — Fill in the three modes

```
modes = {
    "auto": {"type": "auto"},
    "any": {"type": "any"},
    "FORCED": {"type": "tool", "name": "classify_ticket"},
}
```

The classifier tool closes its label space with an `enum` so the category is always one of `order_issue` / `product_question` / `return_request` / `other`:

```
CLASSIFY_TOOL = {
    "name": "classify_ticket",
    "description": "Classify a support ticket into exactly one routing category.",
    "input_schema": {
        "type": "object",
        "properties": {
            "category": {"type": "string",
                "enum": ["order_issue", "product_question", "return_request",
                    "other"]},
            "reason": {"type": "string"},
        },
        "required": ["category", "reason"],
    },
}
```

Step 2 — Run all three modes

```
python exercise_3_tool_choice.py
```

Compare the three blocks: under auto some tickets come back with no classification at all (plain text or a drafted reply); under any a tool is guaranteed but the model may pick the wrong one (`draft_customer_reply`); under FORCED every ticket comes back as a `classify_ticket` call with a category. Only the forced mode is something a routing pipeline can rely on.

Reflection Questions

- auto, any, and FORCED form a spectrum from least to most constrained. Why is “use the narrowest setting that still works” the right default rather than always forcing a tool? What do you lose by over-constraining a turn that genuinely needed to ask a clarifying question?
- In any mode the model is required to call a tool but may pick `draft_customer_reply` instead of `classify_ticket`. For a triage step, why is “called the wrong tool” arguably worse than “called no tool”?
- Forcing `classify_ticket` guarantees the shape of the turn but not the correctness of the category. Where in the pipeline would you put the safety net that catches a confident-but-wrong classification, and what would it look like?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking at your code.

#	Question
1	The model never runs your tool code when it decides which tool to call. Name the three things it actually reads to make that choice.
2	A colleague says “our routing is unreliable, let's upgrade to a bigger model.” Give the technically correct first response.
3	What two boolean-ish fields belong in a structured error envelope, and what does the retry loop do with each?
4	Which HTTP statuses are retryable in this lab, and which two common ones are explicitly NOT?
5	You need a triage step that always returns exactly one classification. Which <code>tool_choice</code> setting do you use, and why not any?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Vague tool names and overlapping descriptions.	search / lookup with “search for stuff” gives the model nothing to disambiguate on. Use object+action names and descriptions that say when NOT to use each tool.
Letting a tool raise an exception to the model.	A raised exception crashes the loop; the model can't reason about it. Always return a structured result and feed it back as a <code>tool_result</code> with <code>is_error</code> set.
Retrying a 404 or 400.	Permanent errors will never succeed — retrying just adds latency and load. Retry only the transient set; stop immediately on permanent ones.

Mistake	Why it matters and what to do instead
Retrying with no backoff or no cap.	Tight retries hammer a struggling service; an uncapped loop can hang forever. Use exponential backoff and a hard attempt cap.
Forcing a tool when the turn needed flexibility.	Over-constraining blocks a legitimate clarifying question. Use the narrowest tool_choice that still does the job — force only the steps that must be deterministic.

6. Quick Reference

6.1 Strong Tool Interface — Skeleton

```
{
  "name": "get_order_status",          # object + action, not a vague verb
  "description": "Retrieve an EXISTING order by ID. Use when the customer "
                "gives an order number. Do NOT use to browse the catalog "
                "— for products use search_products instead.",    # when-to / when-
  NOT
  "input_schema": {
    "type": "object",
    "properties": {
      "order_id": {"type": "string", "pattern": "^NP-[0-9]{6}$"},    # typed +
    constrained
    },
    "required": ["order_id"],
  },
}
```

6.2 Structured Error Envelope + Retry

```
RETRYABLE = {408, 429, 500, 502, 503, 504}

# tool returns data, never raises:
success -> {"isError": False, ...fields}
failure -> {"isError": True, "isRetryable": status in RETRYABLE, "status": ...,
"error": ...}

# loop: retry transient with backoff, cap attempts, stop on permanent
for attempt in range(1, max_attempts + 1):
    r = call_order_tool(id)
    if not r["isError"]: return r
    if r["isRetryable"] and attempt < max_attempts:
        time.sleep(delay); delay *= 2; continue
    return r          # 404 / 400 / out-of-attempts
```

6.3 tool_choice — Modes

```
{"type": "auto"}          # may talk / any tool / no tool
{"type": "any"}           # must call SOME tool (model picks)
{"type": "tool", "name": "classify_ticket"} # must call exactly this tool <-
deterministic
```

```
# Rule: use the NARROWEST setting that still does the job.
```

6.4 File Inventory

File	Exercise	Purpose
<code>exercise_1_tool_interfaces.py</code>	Ex 1	Weak vs. strong toolset + scoring harness over six support questions.
<code>exercise_2_structured_errors.py</code>	Ex 2	isError / isRetryable envelope, retry-with-backoff loop, agentic loop over a flaky Orders service. -- check runs offline.
<code>exercise_3_tool_choice.py</code>	Ex 3	classify_ticket + draft_customer_reply run under auto / any / forced tool_choice.
<code>solutions/</code>	all	Finished reference versions of all three exercise files.

6.5 Further Reading

- Anthropic API — Tool use: <https://docs.claude.com/en/docs/build-with-claude/tool-use>
- Anthropic API — overview: <https://docs.claude.com/en/api/overview>
- Introduction to Model Context Protocol on Skilljar — Defining tools with MCP (S1, S2); MCP clients; Implementing a client (S3)
- Model Context Protocol: Advanced Topics on Skilljar — JSON message types (S2); Introduction to Agent Skills on Skilljar — What are skills?; Skills vs. other Claude Code features (supporting) (S1, S3)